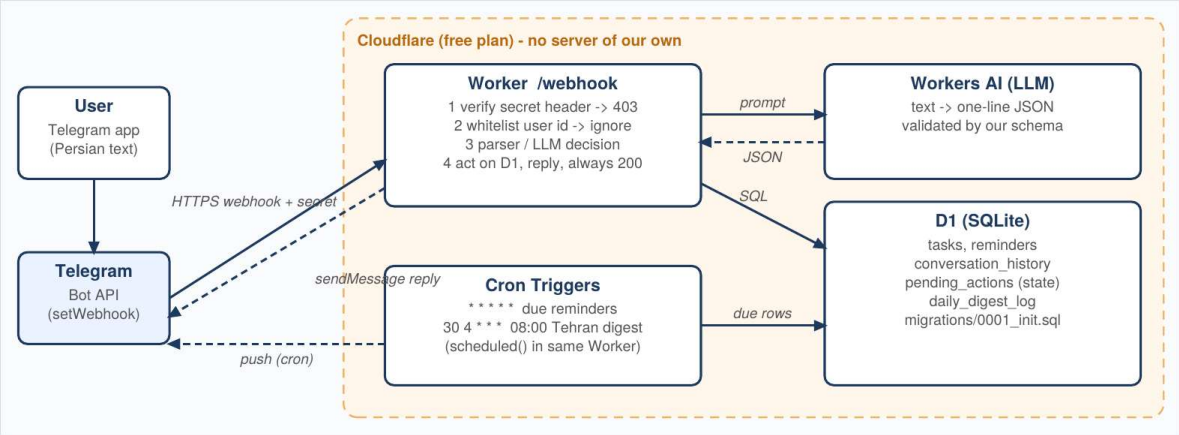
hermes-mini

دستیار بدون‌سرور تلگرام برای مدیریت کارهای روزانه روی Cloudflare

دانشجو	مبینازهانی	**درس**	محاسبات ابری
استادحل تمرین	امیر آبرون	**تاریخ**	۱۵ آبان ۱۴۰۵
آدرس زنده	https://hermes-mini.hermes-mini-tasks.workers.dev/	**مخزن**	https://github.com/lyzohani13-oss/hermes_mini

خلاصه. ربات تلگرامی که پیام‌های آزاد فارسی را می‌فهمد و کارهای روزانه را ثبت، انجام‌شده، حذف و ویرایش می‌کند. تمام سیستم یک Worker، یک پایگاه‌داده‌ی D1 و مدل Workers AI روی پلن رایگان Cloudflare است؛ نه سرور شخصی دارد و نه کانتینر. این گزارش معماری نهایی، تصمیم‌های طراحی، تحلیل هزینه در دو مقیاس و تجربه‌ی رفع مشکل‌ها را شرح می‌دهد.

۱. معماری نهایی و تصمیم‌های طراحی (فاز ۱)



شکل ۱ — معماری نهایی: کاربر ← تلگرام ← Worker ← (Workers AI و D1)، و Cron Triggers برای پیام‌های ارسالی از سمت ربات

تصمیم	انتخاب و توجیه
Webhook یا Polling	**Webhook.** تلگرام هر پیام را به Worker می‌فرستد و کد فقط وقتی پیام هست اجرا می‌شود. Polling به فرایندی نیاز دارد که همیشه زنده باشد (VM یا کانتینر)؛ این هم ممنوع است و هم منابع را هدر می‌دهد. اصالت درخواست با هدر `secret_token` بررسی می‌شود.
چرا FaaS	بار کاری کوچک، منقطع و رویدادمحور است (چند پیام در روز و بقیه‌ی زمان بیکار). FaaS به‌ازای هر اجرا هزینه و مقیاس می‌گیرد، سیستم‌عامل برای نگهداری ندارد و برای «HTTP بگیر، HTTP پاسخ بده» طبیعی است. Cron Triggers زمان‌بندی را بدون سرور اضافه فراهم می‌کند.
چرا D1 و نه KV	داده رابطه‌ای است و با چند کلید جستجو می‌شود: کارها بر اساس کاربر و وضعیت، یادآورها بر اساس `(status, due_at)` و جستجوی عنوان. SQL ایندکس، قید `CHECK` و `INSERT OR IGNORE` (برای یکتایی پیام صبحگاهی) می‌دهد. KV کوئری ندارد و در پلن رایگان فقط ۱٬۰۰۰ نوشتن در روز مجاز است.
وضعیت (state) کجاست	**هیچ‌جا در Worker نیست**؛ ایزوله‌ها بی‌حالت‌اند و هر لحظه ممکن است بازیافت شوند. داده‌ی ماندگار در D1 است؛ حالت کوتاه‌مدت گفتگو («کدام شماره؟»، «همه حذف شوند؟») یک سطر در `pending_actions` با انقضای ۵ دقیقه است؛ حافظه‌ی مدل ۱۰ پیام آخر در `conversation_history` است. به همین دلیل ربات با ری‌استارت از بین نمی‌رود و از هر دستگاهی کار می‌کند.

۲. وب‌هوک امن، مدل داده و زمان تهران (فازهای ۲ و ۳)

- **امنیت وب‌هوک (R5 و R6):** نبود یا نادرستی هدر `X-Telegram-Bot-Api-Secret-Token` پاسخ **۴۰۳** می‌دهد (پیش از خواندن بدنه). درخواست معتبر از کاربر غیرمجاز بی‌صدا کنار گذاشته می‌شود (۲۰۰، بدون ذخیره و بدون پاسخ). مسیر `GET /` برای بررسی سلامت است. پس از احراز هویت، پاسخ **همیشه ۲۰۰** است (خطاها گرفته و لاگ می‌شوند)، وگرنه تلگرام همان پیام را بارها دوباره می‌فرستد.
- **رمزها (R8):** توکن ربات، secret وب‌هوک و شناسه‌ی کاربر مجاز فقط در secret store هستند (`wrangler secret put`). فایل `wrangler.toml` رمزی ندارد و `.dev.vars` در gitignore است.
- **پایگاه‌داده (R7):** پنج جدول در `migrations/0001_init.sql` که با `wrangler d1 migrations apply` اعمال می‌شود؛ در کد هیچ `CREATE TABLE` نیست. ایندکس‌ها: `tasks(user_id, completed, id)`، `reminders(status, due_at)` و `conversation_history(user_id, id)`.
- **تعریف «امروز» به وقت تهران:** Workerها منطقه‌ی زمانی ندارند (UTC). همه‌ی زمان‌های ذخیره‌شده **UTC با قالب ISO-8601** هستند. «روز» برای کاربر، تاریخ تقویمی در `Asia/Tehran` است (UTC+03:30، بدون ساعت تابستانی از ۱۴۰۱) که با `Intl.DateTimeFormat` محاسبه می‌شود. «فردا ساعت ۹» ابتدا به وقت تهران ساخته و سپس به UTC تبدیل می‌شود. cron پیام صبحگاهی `30 4 * * *` است (۰۴:۳۰ UTC برابر ۰۸:۰۰ تهران) و با کلید تاریخ تهران در `daily_digest_log` یکتا می‌شود تا اجرای دوباره‌ی cron پیام را دو بار نفرستد.

۳. لایه‌ی هوشمند (فاز ۴)

طراحی ترکیبی. ربات صرفاً LLM-کند است، برای هر پیام neuron مصرف می‌کند و برای عملیات مخرب ایمن نیست؛ بنابراین تصمیم‌گیری **دو مرحله‌ای** است. مرحله‌ی اول یک تحلیلگر قطعی مبتنی بر «فعل» است که دستورهای بی‌ابهام را اجرا می‌کند (فعل می‌تواند ابتدا یا انتهای جمله باشد؛ کلمه‌های اضافه مثل «کار»، «رو» و «لطفاً» و نیز فاصله و نیم‌فاصله نادیده گرفته می‌شوند). بقیه‌ی پیام‌ها همراه ۱۰ پیام آخر به مدل زبانی (Workers AI) می‌رسد. هر دو مرحله یک مجموعه‌ی عملیات دارند: `add, done, undone, del, edit, search, list, chat`.

- **قرارداد خروجی ساخت‌یافته (R2 و R3):** مدل باید یک شیء JSON بدهد. Worker اولین `{...}` را بیرون می‌کشد (با تحمل code fence و متن اضافه)، آن را parse می‌کند و با **schema خودش اعتبارسنجی** می‌کند: عملیات از فهرست مجاز، رشته‌های با طول محدود، `task_id` عدد صحیح مثبت (ارقام فارسی نرمال می‌شوند) و قواعد معنایی (done/undone/del/edit به شماره یا عنوان نیاز دارند و edit به عنوان جدید).
- **مسیر امن:** JSON خراب، عملیات ناشناخته، خطای فراخوانی مدل یا نقض schema همه به پاسخ `chat` ختم می‌شوند و **هیچ‌چیز در دیتابیس نمی‌نویسند**. خروجی خام شبیه JSON هرگز به کاربر نشان داده نمی‌شود.
- **حفاظ‌ها:** کار جدید **فقط** با فعل صریح «اضافه کن / ثبت کن» ساخته می‌شود و `add` مدل پذیرفته نمی‌شود؛ پس «کارهای خرید رو حذف کن» هرگز کار نمی‌سازد. حذف گروهی را هیچ‌وقت مدل اجرا نمی‌کند و تحلیلگر ابتدا بله/نه می‌پرسد. جستجوی عنوان تقریبی است (فاصله، نیم‌فاصله، حروف عربی/فارسی، همزه)؛ اگر چند کار مشابه باشد، فهرست نشان داده می‌شود، شماره پرسیده می‌شود و سپس عملیات در انتظار اجرا می‌شود.

نمونه‌ی پیام‌های چالشی و نتیجه‌ی آن‌ها:

پیام چالشی	رفتار ربات
«همه‌شون رو حذف کن»	تحلیلگر بله/نه می‌پرسد؛ مسیر مدل برای حذف گروهی مسدود است
«شماره ۳ رو حذف کن» / «کار 3 رو پاک کن»	حذف با شماره؛ اگر وجود نداشت پیام «پیدا نشد»
«کارهای خرید رو حذف کن» (۲ کار مشابه)	فهرست ۲ مورد و پرسش شماره؛ با «لغو» انصراف
«شیر بخر» (بدون فعل اضافه کن)	ثبت نمی‌شود؛ راهنمایی می‌کند که «اضافه کن ...» لازم است
خروجی مدل: متن آزاد، JSON خراب یا عملیات ناشناخته	اعتبارسنجی رد می‌کند؛ پاسخ امن `chat` و دیتابیس دست‌نخورده

۴. عملیات و استقرار (فاز ۵)

- استقرار با `wrangler deploy`؛ در `wrangler.toml` گزینه‌ی `[observability] enabled = true` فعال است و در دمو لاگ زنده با `wrangler tail` نشان داده می‌شود.
- پایگاه‌داده با `wrangler d1 create` ساخته و با `wrangler d1 migrations apply --remote` مایگریت می‌شود؛ وب‌هوک با `setWebhook` و `secret_token` ثبت می‌شود.
- تابع `scheduled` با دو cron اجرا می‌شود: هر دقیقه برای یادآورهای سررسیده و ساعت ۰۴:۳۰ UTC (۰۸:۰۰ تهران) برای پیام صبحگاهی (امتیاز اضافه)؛ هر دو در محدوده‌ی پلن رایگان‌اند.

۵. تحلیل هزینه در دو مقیاس

قیمت‌ها از صفحه‌های رسمی Cloudflare: Workers رایگان ۱۰۰٬۰۰۰ درخواست در روز؛ D1 رایگان ۵ میلیون سطر خواندن و ۱۰۰٬۰۰۰ سطر نوشتن در روز با ۵ گیگابایت فضا؛ Workers AI رایگان ۱۰٬۰۰۰ neuron در روز و سپس ۰٫۰۱۱ دلار به‌ازای هر هزار neuron؛ Workers Paid ماهانه ۵ دلار با ۱۰ میلیون درخواست و ۲۵ میلیارد خواندن و ۵۰ میلیون نوشتن D1 در سهمیه. **فرض‌ها:** ۳۰ پیام در روز برای یک کاربر و ۲۰ پیام برای هر یک از ۱۰٬۰۰۰ کاربر؛ ۳۰٪ پیام‌ها به مدل می‌رسد؛ ۲۵ تا ۱۰۰ neuron در هر فراخوانی (بسته به اندازه‌ی مدل؛ نرخ دقیق را باید از صفحه‌ی مدل انتخابی گرفت)؛ حدود ۳ سطر نوشتن و ۸۰ سطر خواندن برای هر پیام.

منبع در روز	۱ کاربر	۱۰٬۰۰۰ کاربر
درخواست Worker	حدود ۱٬۵۰۰ (۳۰ پیام و ۱٬۴۴۰ cron) از ۱۰۰ هزار رایگان	حدود ۳۰۰ هزار پیام؛ نزدیک ۶ میلیون در ماه. بیش از سقف رایگان؛ پلن Paid ماهی ۵ دلار (۱۰ میلیون در سهمیه)
سطر نوشتن / خواندن D1	حدود ۱۰۰ / ۱۰ هزار (سقف ۱۰۰ هزار / ۵ میلیون)	حدود ۶۰۰ هزار نوشتن در روز (۱۸ میلیون در ماه) و ۱۶ میلیون خواندن؛ داخل سهمیه‌ی Paid
neuron مدل زبانی	حدود ۹ فراخوانی × ۱۰۰ ≈ ۹۰۰ از ۱۰٬۰۰۰ رایگان	۶۰ هزار فراخوانی در روز؛ ۴۵ تا ۱۸۰ میلیون neuron در ماه؛ حدود ۵۰۰ تا ۲٬۰۰۰ دلار
جمع ماهانه	**۰ دلار** (پلن رایگان کافی است)	**حدود ۵۰۰ تا ۲٬۰۰۰ دلار** (۰٫۰۵ تا ۰٫۲۰ دلار برای هر کاربر)؛ بیش از ۹۵٪ آن هزینه‌ی مدل زبانی است

تفسیر. برای یک کاربر همه‌چیز با حاشیه‌ی حدود ۱۰ برابر در پلن رایگان جا می‌شود، پس R8 برقرار است. در ۱۰٬۰۰۰ کاربر، Workers و D1 تقریباً رایگان‌اند و مدل زبانی غالب است؛ برای همین تحلیلگر قطعی یک تصمیم معماری است نه صرفاً بهینه‌سازی (بدون آن هزینه‌ی AI حدود ۳٫۳ برابر می‌شود). در آن مقیاس باید احراز هویت به‌ازای هر کاربر جای شناسه‌ی مجاز واحد را بگیرد، ارسال پیام صبحگاهی با Cloudflare Queues و در چارچوب محدودیت حدود ۳۰ پیام در ثانیه‌ی تلگرام انجام شود و اسکن کارهای هر کاربر `LIMIT` داشته باشد.

۶. کجا گیر کردم و چطور رفعش کردم

- **ربات از هر جمله‌ای کار می‌ساخت.** فال‌بک اول هر متن ناشناخته را کار جدید می‌گرفت و «کارهای خرید رو حذف کن» «اضافه» می‌شد. راه‌حل: ساخت کار فقط با فعل صریح، و رد کردن `add` مدل در کد، نه فقط در پرامپت؛ پرامپت یک درخواست است و اعتبارسنجی یک تضمین.
- **دستورهای regex خیلی سفت بودند.** عبارتی مثل «کار X رو به انجام نشده‌ها برگردون» کار می‌کرد، اما تغییر کوچک در ترتیب یا فاصله جواب نمی‌داد. فهرست قالب‌های جمله با زبان طبیعی جور نمی‌شود؛ پس تحلیلگر را حول «فعلِ ابتدا یا انتهای جمله» و حذف کلمه‌های اضافه بازنویسی کردم و با جدولی از بیش از ۲۰ جمله‌بندی آزمودم.
- **نرمال‌سازی متن فارسی.** نیم‌فاصله، «ی/ک» عربی و فارسی، ارقام فارسی و همزه باعث می‌شد عنوان‌های یکسان متفاوت دیده شوند. راه‌حل: یک تابع نرمال‌سازی برای تشخیص دستور و یک «کلید جستجو»ی جدا که

فاصله، نشانه‌گذاری و تفاوت حروف را حذف می‌کند؛ تطبیق از `SQL LIKE` به جاوااسکریپت منتقل شد، چون SQLite این را نمی‌تواند بیان کند.

- **Worker بی‌حالت و گفتگوی چندمرحله‌ای.** پرسش «کدام یک از این ۳ کار؟» به حافظه بین دو درخواست HTTP نیاز دارد. عملیات در انتظار (همراه عنوان جدید ویرایش) به‌صورت JSON با انقضای ۵ دقیقه در D1 ذخیره می‌شود و پاسخ غیرعددی سؤال قبلی را لغو می‌کند تا کاربر گیر نیفتد.
- **ارسال مجدد تلگرام.** برگرداندن خطا پس از شکست باعث می‌شود تلگرام همان پیام را دوباره بفرستد؛ اکنون Worker پس از احراز هویت ۲۰۰ می‌دهد و خطا را فقط لاگ می‌کند.

۷. جمع‌بندی انطباق با نیازمندی‌ها

نیازمندی	وضعیت و محل پیاده‌سازی
R1 — متن آزاد فارسی، بدون دستور اسلشی	هیچ دستور `/` پردازش نمی‌شود؛ راهنما با متن «راهنما» نمایش داده می‌شود
R2 و R3 — تصمیم مدل، خروجی JSON اعتبارسنجی‌شده، مسیر امن	تحلیلگر قطعی + مدل زبانی، `validateAction` و بازگشت به `chat` (بخش ۳)
R4 — پاسخ گفتگویی با حافظه‌ی کم	۱۰ پیام آخر از `conversation_history`
R5 و R6 — کاربر مجاز و وب‌هوک امن	کاربر غیرمجاز بی‌صدا رد می‌شود؛ بدون secret پاسخ ۴۰۳ (بخش ۲)
R7 — D1 و مایگریشن جدا از کد	`migrations/0001_init.sql`
R8 — فقط پلن رایگان، بدون سرور شخصی	Workers + D1 + Workers AI؛ تحلیل در بخش ۵